

Baker-Go — AI Agent Implementation Plan

Target Agent: Claude / Claude Code

Project Type: Full-Stack Mobile App + Web App (Flutter + Next.js + Node.js + PostgreSQL)

Architecture: Monorepo — shared backend, separate mobile and web frontends

Deployment: Vercel

Languages: English (default), Bahasa Indonesia

Last Updated: May 2026

Table of Contents

1. [Project Overview](#)
 2. [Tech Stack & Prerequisites](#)
 3. [Monorepo Structure](#)
 4. [Step 0 — Environment Setup](#)
 5. [Step 1 — Foundation & Design System](#)
 6. [Step 1.5 — Internationalization \(i18n\)](#)
 7. [Step 2 — Authentication & User Management](#)
 8. [Step 3 — Master Data \(Ingredients & Recipes\)](#)
 9. [Step 4 — HPP Engine & Pricing Strategy](#)
 10. [Step 5 — Daily Transactions & Cash Management](#)
 11. [Step 6 — Analytics Dashboard & Export](#)
 12. [Step 6.5 — Web App \(Next.js\)](#)
 13. [Step 7 — Final QA & Vercel Deployment](#)
 14. [Database Schema Reference](#)
 15. [Design System Reference](#)
-

1. Project Overview

Baker-Go is a production management tool for Indonesian bakery businesses, available as both a **mobile app** (Flutter) and a **web app** (Next.js). Both platforms are built on the same backend and database, giving the owner full access from their phone or a browser. It enables owners to:

- Manage raw ingredients (Master Bahan) and recipes (Master Resep)
- Calculate precise COGS / HPP (Harga Pokok Produksi)
- Determine selling prices for resellers and end users
- Log daily sales (Penjualan) and expenses (Pengeluaran)

- Monitor cash flow via the "Saldo Aman" (Safe Balance) formula
- Generate financial reports and export to PDF/CSV

App language: English (default UI) and Bahasa Indonesia — user can switch in Settings. Code and comments remain in English.

2. Tech Stack & Prerequisites

Layer	Technology	Version
Mobile Frontend	Flutter	≥ 3.19
Web Frontend	Next.js (App Router)	≥ 14
Web Styling	Tailwind CSS + shadcn/ui	Latest
Web State	TanStack Query (React Query)	Latest
Web i18n	next-intl	Latest
Backend Runtime	Node.js	≥ 20 LTS
Backend Framework	Fastify	Latest
ORM	Prisma	≥ 5.x
Database	Neon Serverless Postgres (via Vercel Marketplace)	15+
Auth	JWT (jsonwebtoken)	Latest
Deployment	Vercel	—
Package Manager	pnpm (backend + webapp), pub (Flutter)	—

Required global installations before starting:

```
bash
node --version          # Must be ≥ 20
npm install -g pnpm
dart pub global activate flutterfire_cli
npm install -g vercel
npm install -g prisma
```

Monorepo Structure

All three workspaces live under one `baker-go/` root and are deployed independently to Vercel. The backend is the single source of truth — both frontends consume the same REST API.

```
baker-go/
├── backend/                                ← Fastify API – shared by mobile + web
│   ├── prisma/
│   │   └── schema.prisma
│   ├── src/
│   │   ├── routes/                        ← auth, ingredients, recipes, hpp, transactions, reports
│   │   ├── middleware/
│   │   ├── services/
│   │   └── index.ts
│   ├── vercel.json
│   └── package.json
├── frontend/                              ← Flutter mobile app (iOS + Android)
│   ├── lib/
│   │   ├── core/                         ← theme, router, providers
│   │   ├── features/                     ← auth, dashboard, master, hpp, transactions, reports
│   │   └── main.dart
│   ├── l10n.yaml
│   └── pubspec.yaml
├── webapp/                                ← Next.js web app (same features, browser)
│   ├── app/                              ← App Router pages
│   │   ├── (auth)/                       ← login, register
│   │   ├── (dashboard)/                  ← home, master-bahan, master-resep, hpp, etc.
│   │   └── layout.tsx
│   ├── components/                       ← shared UI components
│   ├── lib/                              ← API client, utils, i18n
│   ├── messages/                         ← en.json + id.json (next-intl)
│   ├── public/
│   ├── tailwind.config.ts
│   ├── next.config.ts
│   └── package.json
```

How the three workspaces relate:

	Backend	Flutter	Next.js
Language	TypeScript	Dart	TypeScript
Talks to	Neon Postgres (Prisma)	Backend REST API	Backend REST API
Auth	Issues JWT	Stores JWT (secure storage)	Stores JWT (httpOnly cookie)
Deployed on	Vercel (serverless)	App stores / APK	Vercel (edge)
i18n	<code>?lang=</code> query param	ARB files + LocaleProvider	next-intl + JSON messages

Step 0 — Environment Setup

Objective: Configure all tools, credentials, and folder structure before writing a single line of app code.

0.1 Create Project Root

```
bash

mkdir baker-go && cd baker-go
git init
echo "node_modules/\n.env\n.vercel\nbuild/\n.dart_tool/" > .gitignore
```

0.2 Neon Database — Provision via Vercel Marketplace

Note: Vercel no longer runs its own Postgres. The official database is now **Neon Serverless Postgres**, provisioned through the Vercel Marketplace. It is free to use (no credit card required) on the Hobby plan with 0.5 GB storage and 100 compute-unit hours/month — more than enough for Baker-Go.

1. Log in to vercel.com → go to your project → **Storage** tab
2. Click **Connect Store** → select **Neon Postgres** from the Marketplace
3. Choose **Create New Neon Account** (or link existing) → name the database `baker-go-db`
4. Select your region (e.g., `sin1` for Singapore, closest to Indonesia)
5. After creation, go to the `.env.local` tab in the Neon integration panel and copy all environment variables
6. Save these values — you will need them in Step 0.4

Free tier limits (Neon via Vercel):

Resource	Free Limit
Storage	0.5 GB per project
Compute	100 CU-hours/month
Projects	Up to 3
Branching	Unlimited (within compute)
Credit card	Not required

0.3 Backend Scaffold

```
bash

mkdir backend && cd backend
pnpm init
pnpm add fastify @fastify/jwt @fastify/cors @prisma/client dotenv
pnpm add -D prisma typescript ts-node @types/node nodemon
npx prisma init --datasource-provider postgresql
```

0.4 Configure Backend `.env`

Create `backend/.env` with the values from the Neon integration panel in Vercel:

```
env

# Neon Serverless Postgres (via Vercel Marketplace)
DATABASE_URL="postgresql://user:password@ep-xxx.sin1.neon.tech/neondb?sslmode=require"
DIRECT_URL="postgresql://user:password@ep-xxx.sin1.neon.tech/neondb?sslmode=require"
JWT_SECRET="your-super-secret-key-change-this"
PORT=3000
NODE_ENV=development
```

Tip: Neon provides both a **pooled connection string** (use for `DATABASE_URL` — handles concurrent requests) and a **direct connection string** (use for `DIRECT_URL` — required by Prisma migrate). Both are shown in the Vercel Storage → your database → `.env.local` tab.

0.5 Flutter Scaffold

```
bash

cd ..
flutter create --org com.bakergo --project-name baker_go frontend
cd frontend
```

Add these packages to `frontend/pubspec.yaml`:

```
yaml
dependencies:
  flutter:
    sdk: flutter
  flutter_localizations:      # built-in Flutter i18n support
    sdk: flutter
  dio: ^5.4.0                 # HTTP client
  flutter_secure_storage: ^9.0.0 # JWT token storage
  go_router: ^13.0.0          # Navigation
  provider: ^6.1.0            # State management
  fl_chart: ^0.67.0           # Charts for dashboard
  intl: ^0.19.0               # Number/date formatting (Rupiah) + ARB support
  cached_network_image: ^3.3.0
  shimmer: ^3.0.0             # Loading skeletons

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^3.0.0

flutter:
  generate: true              # Required for auto-generated l10n code
```

Run `flutter pub get` to install.

0.6 Vercel Project Linking

```
bash
cd ../backend
vercel link      # Connect to your Vercel account and project
vercel env pull  # Pull environment variables to .env.local
```

0.7 Final Folder Structure

```
baker-go/
├─ backend/
│   ├─ prisma/
│   │   └─ schema.prisma
│   ├─ src/
│   │   ├─ routes/
│   │   ├─ middleware/
│   │   ├─ services/
│   │   └─ index.ts
```

```
|   ├── vercel.json
|   ├── .env
|   └── package.json
├── frontend/                ← Flutter mobile
|   ├── lib/
|   |   ├── core/
|   |   ├── features/
|   |   └── main.dart
|   ├── l10n.yaml
|   └── pubspec.yaml
├── webapp/                  ← Next.js web (scaffolded in Step 6.5)
|   ├── app/
|   ├── components/
|   ├── lib/
|   ├── messages/
|   ├── tailwind.config.ts
|   └── package.json
└── .gitignore
```

Agent Prompt for Step 0:

"Claude, start Step 0. Create the project folder structure exactly as specified in the plan. Scaffold the Node.js/Fastify backend with Prisma and connect it to **Neon Postgres via Vercel Marketplace** using the DATABASE_URL and DIRECT_URL from the Neon integration panel. Make sure the Prisma schema uses both `url` (pooled) and `directUrl` (direct) connection strings as required by Neon. Scaffold the Flutter project with all dependencies listed. Verify the database connection by running `npm run prisma db pull` (or an empty migration) and confirm it succeeds without errors."

Step 1 — Foundation & Design System

Objective: Initialize both projects and establish a consistent visual language derived from the Baker-Go design reference.

1.1 Prisma — Initial Schema Setup

In `backend/prisma/schema.prisma`, configure the generator and datasource for Neon:

```
prisma
```

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL") // pooled connection (Neon)
  directUrl = env("DIRECT_URL")  // direct connection for migrations
}
```

Why two URLs? Neon uses connection pooling by default for serverless environments. Prisma requires the `directUrl` (non-pooled) specifically for running migrations — regular queries use the pooled `DATABASE_URL`.

Run first migration:

```
bash

npx prisma migrate dev --name init
npx prisma generate
```

1.2 Flutter — Design System (`ThemeData`)

Create `frontend/lib/core/theme/app_theme.dart`:

```
dart
```



```

import 'package:flutter/material.dart';

class AppColors {
  // Primary palette (extracted from Baker-Go design)
  static const Color primary      = Color(0xFFA0813A); // warm brown/gold
  static const Color primaryDark  = Color(0xFF7A5F28);
  static const Color primaryLight = Color(0xFFD4AA6F);
  static const Color background   = Color(0xFFFF5F0E8); // warm off-white
  static const Color surface      = Color(0xFFFFFFFF);
  static const Color cardBg       = Color(0xFFEDE8DF);

  // Text
  static const Color textPrimary   = Color(0xFF2C2C2C);
  static const Color textSecondary = Color(0xFF8A8A8A);
  static const Color textOnPrimary = Color(0xFFFFFFFF);

  // Semantic
  static const Color success       = Color(0xFF4CAF50);
  static const Color error         = Color(0xFFE53935);
  static const Color warning       = Color(0xFFFFA726);
}

class AppTypography {
  static const String fontFamily = 'Inter'; // or system default

  static const TextStyle displayLarge  = TextStyle(fontSize: 28, fontWeight: FontWeight.w900);
  static const TextStyle headlineMedium = TextStyle(fontSize: 22, fontWeight: FontWeight.w900);
  static const TextStyle titleMedium   = TextStyle(fontSize: 16, fontWeight: FontWeight.w900);
  static const TextStyle bodyMedium    = TextStyle(fontSize: 14, fontWeight: FontWeight.w900);
  static const TextStyle labelSmall    = TextStyle(fontSize: 12, fontWeight: FontWeight.w900);
}

ThemeData buildAppTheme() {
  return ThemeData(
    useMaterial3: true,
    colorScheme: ColorScheme.fromSeed(
      seedColor: AppColors.primary,
      background: AppColors.background,
      surface: AppColors.surface,
    ),
    scaffoldBackgroundColor: AppColors.background,
    appBarTheme: const AppBarTheme(
      backgroundColor: AppColors.background,
      foregroundColor: AppColors.textPrimary,
      elevation: 0,
    ),
  );
}

```

```

elevatedButtonTheme: ElevatedButtonThemeData(
  style: ElevatedButton.styleFrom(
    backgroundColor: AppColors.primary,
    foregroundColor: AppColors.textOnPrimary,
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
    minimumSize: const Size(double.infinity, 50),
  ),
),
inputDecorationTheme: InputDecorationTheme(
  filled: true,
  fillColor: AppColors.cardBg,
  border: OutlineInputBorder(
    borderRadius: BorderRadius.circular(12),
    borderSide: BorderSide.none,
  ),
  contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
),
cardTheme: CardTheme(
  color: AppColors.surface,
  elevation: 0,
  shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
),
);
}

```

Apply in `main.dart`:

```

dart

MaterialApp.router(
  title: 'BakersGo',
  theme: buildAppTheme(),
  routerConfig: appRouter,
  localizationsDelegates: AppLocalizations.localizationsDelegates,
  supportedLocales: AppLocalizations.supportedLocales,
  locale: context.watch<LocaleProvider>().locale, // driven by user preference
);

```

1.3 Navigation Setup with GoRouter

Create `frontend/lib/core/router/app_router.dart` with the following routes:

Route	Screen
/	Splash / Onboarding
/login	Sign In
/register	Sign Up
/home	Dashboard (Beranda)
/profile	Profil
/master-bahan	Master Bahan
/master-resep	Master Resep
/hpp	HPP Calculator
/pengeluaran	Expenses
/penjualan	Sales
/laporan	Reports

Agent Prompt for Step 1:

"Claude, start Step 1. Set up the Flutter design system using the exact color values from the design file: primary brown-gold (#A0813A), off-white background (#F5F0E8), card background (#EDE8DF). Create AppColors, AppTypography, and buildAppTheme() in lib/core/theme/app_theme.dart. Configure GoRouter with all routes listed in the implementation plan. Scaffold placeholder screens for each route so navigation works end-to-end immediately. Also enable generate: true in pubspec.yaml and add flutter_localizations as a dependency — i18n will be wired up in Step 1.5."

Step 1.5 — Internationalization (i18n)

Objective: Enable bilingual support (English default + Bahasa Indonesia) using Flutter's official flutter_localizations + ARB file approach. The user selects their preferred language in the Profile/Settings screen, and the choice is persisted across sessions.

1.5.1 i10n Configuration

Create frontend/i10n.yaml in the Flutter project root:

```
yaml
```

```
arb-dir: lib/l10n
template-arb-file: app_en.arb
output-localization-file: app_localizations.dart
output-class: AppLocalizations
```

This tells Flutter's code generator where to find the translation files and what class name to output.

1.5.2 ARB Translation Files

Create `frontend/lib/l10n/app_en.arb` (English — default):

json

```
{
  "@@locale": "en",

  "appName": "BakersGo",
  "onboardingTitle": "Welcome to BakersGo!",
  "onboardingSubtitle": "Level up your bakery, starting now!",
  "onboardingCta": "Get Started!",

  "signIn": "Sign In",
  "signUp": "Sign Up",
  "email": "Email",
  "password": "Password",
  "userId": "User ID",
  "brandName": "Brand Name",
  "save": "Save",
  "cancel": "Cancel",
  "edit": "Edit",
  "delete": "Delete",
  "resetPassword": "Reset Password",
  "oldPassword": "Old Password",
  "newPassword": "New Password",

  "home": "Home",
  "profile": "Profile",
  "masterIngredients": "Master Ingredients",
  "masterRecipes": "Master Recipes",
  "hpp": "HPP",
  "expenses": "Expenses",
  "sales": "Sales",
  "reports": "Reports",

  "sales_amount": "Sales",
  "expenses_amount": "Expenses",
  "safeBalance": "Safe Balance",
  "realBalance": "Real Balance",

  "ingredientName": "Ingredient Name",
  "unit": "Unit",
  "packagePrice": "Package Price (Rp)",
  "packageVolume": "Package Volume",
  "unitPrice": "Unit Price",

  "recipeName": "Recipe Name",
  "servings": "Servings",
  "baseRecipeCost": "Base Recipe Cost",
  "addIngredient": "Add Ingredient",
```

```
"amount": "Amount",

"productName": "Product Name",
"batchSize": "Batch Size (units)",
"kitchenZone": "Kitchen Zone",
"packagingZone": "Packaging Zone",
"pricingZone": "Pricing Zone",
"electricity": "Electricity (Rp)",
"gas": "Gas (Rp)",
"labor": "Labor (Rp)",
"overhead": "Overhead (Rp)",
"box": "Box / Packaging (Rp)",
"sticker": "Sticker (Rp)",
"disposable": "Disposable (Rp)",
"resellerMargin": "Reseller Margin (%)",
"endUserMargin": "End User Margin (%)",
"resellerPrice": "Reseller Price",
"endUserPrice": "End User Price",
"resellerProfit": "Reseller Profit",
"endUserProfit": "End User Profit",
"totalCogs": "Total COGS (HPP)",

"channel": "Channel",
"reseller": "Reseller",
"endUser": "End User",
"quantity": "Quantity",
"total": "Total",
"date": "Date",
"description": "Description",
"category": "Category",

"filterByDay": "Day",
"filterByMonth": "Month",
"filterByYear": "Year",
"exportCsv": "Export CSV",
"exportPdf": "Export PDF",

"language": "Language",
"languageEnglish": "English",
"languageIndonesian": "Bahasa Indonesia",

"errorGeneric": "Something went wrong. Please try again.",
"errorInvalidCredentials": "Invalid email/ID or password.",
"successSaved": "Saved successfully.",
"emptyState": "Nothing here yet. Tap + to add."
```

```
}
```

Create frontend/lib/l10n/app_id.arb (Bahasa Indonesia):

json

```
{
  "@@locale": "id",

  "appName": "BakersGo",
  "onboardingTitle": "Selamat datang di BakersGo!",
  "onboardingSubtitle": "Saatnya naik kelas dari sekarang!",
  "onboardingCta": "Lanjutkan!",

  "signIn": "Masuk",
  "signUp": "Daftar",
  "email": "Email",
  "password": "Kata Sandi",
  "userId": "ID Pengguna",
  "brandName": "Nama Brand",
  "save": "Simpan",
  "cancel": "Batal",
  "edit": "Edit",
  "delete": "Hapus",
  "resetPassword": "Reset Sandi",
  "oldPassword": "Kata Sandi Lama",
  "newPassword": "Kata Sandi Baru",

  "home": "Beranda",
  "profile": "Profil",
  "masterIngredients": "Master Bahan",
  "masterRecipes": "Master Resep",
  "hpp": "HPP",
  "expenses": "Pengeluaran",
  "sales": "Penjualan",
  "reports": "Laporan",

  "sales_amount": "Penjualan",
  "expenses_amount": "Pengeluaran",
  "safeBalance": "Saldo Aman",
  "realBalance": "Saldo Nyata",

  "ingredientName": "Nama Bahan",
  "unit": "Satuan",
  "packagePrice": "Harga Kemasan (Rp)",
  "packageVolume": "Volume Kemasan",
  "unitPrice": "Harga Satuan",

  "recipeName": "Nama Resep",
  "servings": "Porsi",
  "baseRecipeCost": "Biaya Dasar Resep",
  "addIngredient": "Tambah Bahan",
```



```
"amount": "Jumlah",

"productName": "Nama Produk",
"batchSize": "Jumlah Produksi (pcs)",
"kitchenZone": "Zona Dapur",
"packagingZone": "Zona Final",
"pricingZone": "Zona Penjualan",
"electricity": "Listrik (Rp)",
"gas": "Gas (Rp)",
"labor": "Tenaga Kerja (Rp)",
"overhead": "Overhead (Rp)",
"box": "Kotak / Kemasan (Rp)",
"sticker": "Stiker (Rp)",
"disposable": "Disposable (Rp)",
"resellerMargin": "Margin Reseller (%)",
"endUserMargin": "Margin End User (%)",
"resellerPrice": "Harga Reseller",
"endUserPrice": "Harga End User",
"resellerProfit": "Profit Reseller",
"endUserProfit": "Profit End User",
"totalCogs": "HPP per Unit",

"channel": "Saluran",
"reseller": "Reseller",
"endUser": "End User",
"quantity": "Jumlah",
"total": "Total",
"date": "Tanggal",
"description": "Keterangan",
"category": "Kategori",

"filterByDay": "Hari",
"filterByMonth": "Bulan",
"filterByYear": "Tahun",
"exportCsv": "Ekspor CSV",
"exportPdf": "Ekspor PDF",

"language": "Bahasa",
"languageEnglish": "English",
"languageIndonesian": "Bahasa Indonesia",

"errorGeneric": "Terjadi kesalahan. Silakan coba lagi.",
"errorInvalidCredentials": "Email/ID atau kata sandi salah.",
"successSaved": "Berhasil disimpan.",
"emptyState": "Belum ada data. Ketuk + untuk menambah."
```

```
}
```

Run the code generator after creating these files:

```
bash

flutter gen-l10n
# Output: lib/generated/l10n/app_localizations.dart (auto-generated, do not edit)
```

1.5.3 LocaleProvider — State Management

Create `frontend/lib/core/providers/locale_provider.dart`:

```
dart

import 'package:flutter/material.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

class LocaleProvider extends ChangeNotifier {
  static const _key = 'app_locale';
  final _storage = const FlutterSecureStorage();

  Locale _locale = const Locale('en'); // English default

  Locale get locale => _locale;

  Future<void> load() async {
    final saved = await _storage.read(key: _key);
    if (saved != null) {
      _locale = Locale(saved);
      notifyListeners();
    }
  }

  Future<void> setLocale(Locale locale) async {
    _locale = locale;
    await _storage.write(key: _key, value: locale.languageCode);
    notifyListeners();
  }
}
```

Register in `main.dart`:

```
dart
```

```

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final localeProvider = LocaleProvider();
  await localeProvider.load(); // restore saved language on startup

  runApp(
    ChangeNotifierProvider(
      create: (_) => localeProvider,
      child: const BakersGoApp(),
    ),
  );
}

```

1.5.4 Language Switcher UI

Add a language selector to the **Profile screen** (and optionally the Onboarding screen for first-time setup):

```

dart

// In profile_screen.dart
final localeProvider = context.watch<LocaleProvider>();
final l10n = AppLocalizations.of(context)!;

ListTile(
  leading: const Icon(Icons.language),
  title: Text(l10n.language),
  trailing: DropdownButton<Locale>(
    value: localeProvider.locale,
    items: const [
      DropdownMenuItem(value: Locale('en'), child: Text('English')),
      DropdownMenuItem(value: Locale('id'), child: Text('Bahasa Indonesia')),
    ],
    onChanged: (locale) {
      if (locale != null) localeProvider.setLocale(locale);
    },
  ),
);

```

1.5.5 Using Translations in Screens

Replace all hardcoded strings with `AppLocalizations` keys:

```

dart

```

```
// ❌ Before
Text('Master Ingredients')
TextFormField(labelText: 'Ingredient Name')

// ✅ After
final l10n = AppLocalizations.of(context)!;
Text(l10n.masterIngredients)
TextFormField(decoration: InputDecoration(labelText: l10n.ingredientName))
```

1.5.6 PDF Export — Bilingual Reports

The PDF export (Step 6) must also respect the user's language setting. Pass the active locale to the backend export endpoint:

```
GET /reports/export/pdf?from=&to=&lang=en // English report
GET /reports/export/pdf?from=&to=&lang=id // Bahasa Indonesia report
```

The backend reads the `(lang)` query param and switches report labels accordingly using a simple translation map in `(src/utils/report_i18n.ts)`:

```
typescript

const reportLabels = {
  en: {
    title: 'Financial Report',
    totalIncome: 'Total Income',
    totalExpenses: 'Total Expenses',
    netProfit: 'Net Profit',
    safeBalance: 'Safe Balance',
    date: 'Date', product: 'Product', channel: 'Channel', qty: 'Qty', amount: 'Amount',
    category: 'Category', description: 'Description',
  },
  id: {
    title: 'Laporan Keuangan',
    totalIncome: 'Total Penjualan',
    totalExpenses: 'Total Pengeluaran',
    netProfit: 'Laba Bersih',
    safeBalance: 'Saldo Aman',
    date: 'Tanggal', product: 'Produk', channel: 'Saluran', qty: 'Jml', amount: 'Jumlah',
    category: 'Kategori', description: 'Keterangan',
  },
};
```

Agent Prompt for Step 1.5:

"Claude, proceed to Step 1.5. Set up Flutter i18n using the official flutter_localizations + ARB approach. Create l10n.yaml in the project root with generate: true. Create lib/l10n/app_en.arb (English, default) and lib/l10n/app_id.arb (Bahasa Indonesia) with all string keys listed in the implementation plan. Run flutter gen-l10n to generate the AppLocalizations class. Create LocaleProvider using ChangeNotifier and persist the user's language choice with flutter_secure_storage. Add a language switcher dropdown to the Profile screen with two options: English and Bahasa Indonesia. Replace all hardcoded UI strings across every screen with AppLocalizations.of(context)! keys. Default language must be English. On the backend, add a ?lang=en|id query parameter to the PDF export endpoint and switch report column labels accordingly."

Step 2 — Authentication & User Management

Objective: Secure the app with JWT-based authentication, covering Sign In, Sign Up, and Profile management.

2.1 Backend — Auth API Endpoints

Method	Endpoint	Description
POST	/auth/register	Create new user account
POST	/auth/login	Login, return JWT
GET	/auth/profile	Get current user (protected)
PUT	/auth/profile	Update brand name / user ID
PUT	/auth/password	Change password

2.2 Prisma — User Model

prisma

```

model User {
  id          String    @id @default(cuid())
  email       String    @unique
  userId      String    @unique // @UserID shown in UI
  brandName   String    // Nama Brand / Bakery
  password    String    // bcrypt hashed
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  ingredients Ingredient[]
  recipes       Recipe[]
  transactions  Transaction[]
  expenses      Expense[]
}

```

2.3 Backend — Auth Logic (Fastify + JWT)

Install: `pnpm add bcryptjs` and `pnpm add -D @types/bcryptjs`

Key logic in `src/routes/auth.ts`:

```

typescript

// Register
const hashed = await bcrypt.hash(password, 10);
const user = await prisma.user.create({ data: { email, userId, brandName, password:
const token = fastify.jwt.sign({ id: user.id }, { expiresIn: '30d' });
return { token, user: { id: user.id, email, userId, brandName } };

// Login
const valid = await bcrypt.compare(password, user.password);
if (!valid) throw new Error('Invalid credentials');
const token = fastify.jwt.sign({ id: user.id }, { expiresIn: '30d' });

```

2.4 Flutter — Auth Screens

Implement these screens matching the design reference:

Screen: Onboarding (page 1)

- Full-screen warm brown background with baking illustration pattern
- White card bottom sheet: "Selamat datang di BakersGo!" + "Lanjutkan!" FAB button

Screen: Sign In / Sign Up selector (page 2)

- Header: "Halo, Bakers!"

- Two large buttons: "Sign In" and "Sign Up"

Screen: Sign In form (page 3)

- Fields: Email / User ID, Password
- Primary CTA: "Sign In" button

Screen: Sign Up form (pages 6-7)

- Fields: Email, Password, User ID
- Primary CTA: "Sign Up" button

Screen: Welcome (page 8)

- "Selamat Datang, @UserID" confirmation screen after registration

Screen: Profile (page 4 — view, page 5 — edit)

- View: @Nama Brand, @UserID, Email, "Reset Sandi" button, "Edit" button
- Edit: Editable fields, "Simpan" button
- Change Password: Kata sandi lama, Kata sandi baru, "Simpan" button (page 10)

2.5 Flutter — Auth Service

Create `lib/features/auth/services/auth_service.dart`:

```
dart

class AuthService {
  final Dio _dio;

  Future<String> login(String emailOrUserId, String password) async { ... }
  Future<String> register(String email, String password, String userId, String brand) async { ... }
  Future<void> saveToken(String token) async { /* flutter_secure_storage */ }
  Future<String?> getToken() async { ... }
  Future<void> logout() async { ... }
}
```

Agent Prompt for Step 2:

"Claude, proceed to Step 2. Implement JWT authentication in the Fastify backend with register and login endpoints using bcrypt for password hashing. In Flutter, build the Onboarding, Auth Selector, Sign In, Sign Up, and Welcome screens matching the Baker-Go design reference. Use flutter_secure_storage to persist the JWT token. Implement a GoRouter redirect guard that sends unauthenticated users to `/login` and authenticated

users to . Match the warm brown color scheme and rounded input field styling exactly."

Step 3 — Master Data (Ingredients & Recipes)

Objective: Build the single source of truth for all raw materials and recipes with automatic cost calculations.

3.1 Prisma — Models

prisma


```

model Ingredient {
  id          String    @id @default(cuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id])
  name        String    // Nama bahan (e.g., "Tepung Terigu")
  unit        String    // Satuan (e.g., "gram", "ml", "butir")
  packagePrice Decimal   // Harga per kemasan
  packageVolume Decimal  // Volume per kemasan
  unitPrice   Decimal   // AUTO: packagePrice / packageVolume
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  recipeItems RecipeItem[]
}

model Recipe {
  id          String    @id @default(cuid())
  userId      String
  user        User      @relation(fields: [userId], references: [id])
  name        String    // Nama resep
  servings    Int       @default(1)
  baseRecipeCost Decimal // AUTO: sum of (amount × unitPrice)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  items       RecipeItem[]
  hppEntries  HppEntry[]
}

model RecipeItem {
  id          String    @id @default(cuid())
  recipe      Recipe    @relation(fields: [recipeId], references: [id], onDelete:
  recipeId    String
  ingredient   Ingredient @relation(fields: [ingredientId], references: [id])
  ingredientId String
  amount       Decimal   // grams / ml / units used
  itemCost     Decimal   // AUTO: amount × ingredient.unitPrice
}

```

3.2 Business Logic

Unit Price Formula:

$$\text{unitPrice} = \text{packagePrice} / \text{packageVolume}$$

Calculated server-side on every ingredient create/update. Never rely on client-sent `unitPrice`.

Base Recipe Cost Formula:

```
baseRecipeCost = Σ (recipeItem.amount × ingredient.unitPrice)
```

Recalculated on the server every time a recipe item is added, updated, or deleted.

3.3 Backend — API Endpoints

Ingredients:

Method	Endpoint	Description
GET	/ingredients	List all (current user)
POST	/ingredients	Create + auto-calc unitPrice
PUT	/ingredients/:id	Update + recalc unitPrice
DELETE	/ingredients/:id	Delete (check for recipe references)

Recipes:

Method	Endpoint	Description
GET	/recipes	List all
POST	/recipes	Create recipe
GET	/recipes/:id	Get with all items and live costs
PUT	/recipes/:id	Update
DELETE	/recipes/:id	Delete
POST	/recipes/:id/items	Add ingredient to recipe
PUT	/recipes/:id/items/:itemId	Update item amount
DELETE	/recipes/:id/items/:itemId	Remove ingredient

3.4 Flutter — Screens

Master Bahan screen:

- List of ingredients with name, unit, and unit price (formatted as Rp)

- FAB to add new ingredient
- Form: Nama Bahan, Satuan, Harga Kemasan (Rp), Volume Kemasan → shows calculated Harga Satuan

Master Resep screen:

- List of recipes with name and base cost
- Recipe detail: list of ingredients with amounts
- Form to add ingredients from a searchable dropdown of Master Bahan

Agent Prompt for Step 3:

"Claude, proceed to Step 3. Build the Master Bahan (Ingredients) and Master Resep (Recipes) modules. On the backend, ensure `unitPrice` is always auto-calculated as $\text{packagePrice} / \text{packageVolume}$ — never accept it from the client. When a recipe is saved or modified, recalculate `baseRecipeCost` by querying the latest ingredient unit prices. In Flutter, build list screens with a FAB for adding new items and detail screens for viewing/editing. Format all currency values as Indonesian Rupiah (e.g., Rp 12.500)."

Step 4 — HPP Engine & Pricing Strategy

Objective: Build the multi-zone cost calculation engine to determine total COGS and selling price margins.

4.1 Prisma — HPP Model

prisma

```

model HppEntry {
  id                String    @id @default(cuid())
  userId            String
  recipe            Recipe    @relation(fields: [recipeId], references: [id])
  recipeId          String
  productName       String    // Final product name
  batchSize         Int       // Units produced per batch

  // Zona Dapur (Kitchen Zone)
  electricityCost   Decimal   @default(0)
  gasCost           Decimal   @default(0)
  laborCost         Decimal   @default(0)
  overheadCost      Decimal   @default(0)

  // Zona Final (Packaging Zone)
  boxCost           Decimal   @default(0)
  stickerCost       Decimal   @default(0)
  disposableCost    Decimal   @default(0)

  // Zona Penjualan (Selling Zone)
  resellerMargin    Decimal   @default(0) // percentage
  endUserMargin     Decimal   @default(0) // percentage

  // Calculated (stored for fast read, recalculated on save)
  totalCogs         Decimal   // baseRecipeCost + all costs above / batchSize
  resellerPrice      Decimal   // totalCogs × (1 + resellerMargin/100)
  endUserPrice       Decimal   // totalCogs × (1 + endUserMargin/100)
  resellerProfit     Decimal   // resellerPrice - totalCogs
  endUserProfit      Decimal   // endUserPrice - totalCogs

  createdAt         DateTime @default(now())
  updatedAt         DateTime @updatedAt
}

```

4.2 HPP Calculation Formula

```

totalCogs per unit = (
  recipe.baseRecipeCost
  + electricityCost + gasCost + laborCost + overheadCost
  + boxCost + stickerCost + disposableCost
) / batchSize

resellerPrice = totalCogs × (1 + resellerMargin / 100)
endUserPrice  = totalCogs × (1 + endUserMargin / 100)

```

```
resellerProfit = resellerPrice - totalCogs
endUserProfit  = endUserPrice  - totalCogs
```

4.3 Flutter — Multi-Step HPP Form

Implement as a stepper widget with 3 zones:

Zone 1 — Zona Dapur:

- Select recipe (dropdown from Master Resep, shows base cost)
- Input: Batch size (number of units produced)
- Inputs: Listrik (electricity), Gas, Tenaga Kerja (labor), Overhead
- Shows: Biaya Dapur subtotal

Zone 2 — Zona Final (Packaging):

- Inputs: Kotak/Box, Stiker, Disposable
- Shows: Biaya Packaging subtotal

Zone 3 — Zona Penjualan:

- Inputs: Margin Reseller (%), Margin End User (%)
- Shows live preview:
 - HPP per unit
 - Harga Reseller / Profit Reseller
 - Harga End User / Profit End User
- "Simpan" to save the HPP entry

Agent Prompt for Step 4:

"Claude, proceed to Step 4. Build the HPP Engine. Create a 3-zone stepper form in Flutter: Zona Dapur (kitchen costs), Zona Final (packaging costs), and Zona Penjualan (pricing). All calculation logic must run on the backend — the Flutter client sends the raw inputs and receives back the calculated `totalCogs`, `resellerPrice`, `endUserPrice`, `resellerProfit`, and `endUserProfit`. Show a live calculation preview in the Zona Penjualan step as the user types margin values. Validate that batchSize is > 0 before allowing submission."

Step 5 — Daily Transactions & Cash Management

Objective: Enable daily sales logging and implement the "Saldo Aman" (Safe Balance) cash management formula.

prisma

```
model Transaction {
  id          String   @id @default(cuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id])
  date        DateTime @default(now())
  type        String   // "SALE" | "EXPENSE"
  description String
  amount      Decimal
 _hppEntryId String?   // optional link to HPP product (for sales)
  quantity    Int?     // units sold
  channel      String? // "RESELLER" | "END_USER"
  category     String? // expense category label
  createdAt   DateTime @default(now())
}

model Expense {
  id          String   @id @default(cuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id])
  date        DateTime @default(now())
  description String
  amount      Decimal
  category     String // "ELECTRICITY" | "TITHE" | "CAPITAL" | "OTHER"
  createdAt   DateTime @default(now())
}
```

5.2 Saldo Aman Formula

```
totalIncome      =  $\Sigma$  sales transactions this period
totalExpenses    =  $\Sigma$  expense transactions this period
realBalance      = totalIncome - totalExpenses

// Reserved deductions
electricityReserve = projected monthly electricity cost
titheAmount        = totalIncome  $\times$  0.025 // 2.5% zakat/sedekah
capitalReserve     = totalIncome  $\times$  0.10  // 10% capital reserve (configurable)

safeBalance = realBalance - (electricityReserve + titheAmount + capitalReserve)
```

UX Note: Show `safeBalance` prominently on the dashboard. This is the amount the owner can safely use as personal income or dividends.

5.3 Backend — API Endpoints

Method	Endpoint	Description
GET	/transactions	List with date filter {from=&to=}
POST	/transactions/sale	Log a sale
POST	/transactions/expense	Log an expense
DELETE	/transactions/:id	Delete a transaction
GET	/dashboard/summary	Total income, expenses, safe balance

5.4 Flutter — Transaction Screens

Penjualan (Sales) screen:

- Date picker for the transaction
- Select product from HPP list (auto-fills price from resellerPrice or endUserPrice)
- Input: Quantity sold
- Select channel: Reseller / End User
- Total is calculated automatically
- List of today's / recent sales

Pengeluaran (Expenses) screen:

- Input: Description, amount, category
- Category options: Listrik, Gas, Tenaga Kerja, Bahan Baku, Lain-lain
- List of recent expenses

Agent Prompt for Step 5:

"Claude, proceed to Step 5. Build the Penjualan (Sales) and Pengeluaran (Expenses) transaction modules. When logging a sale, allow the user to select from the HPP product list — automatically populate the selling price based on the chosen channel (Reseller or End User). Implement the /dashboard/summary endpoint that returns {totalIncome}, {totalExpenses}, {realBalance}, and {safeBalance} using the Saldo Aman formula. Display these on the Beranda (Home) screen with the monthly period selector matching the design."

Step 6 — Analytics Dashboard & Export

Objective: Visualize business health with charts and enable financial report exports.

6.1 Dashboard (Beranda) — Home Screen

Matching design reference page 9/11:

- **Header:** "Halo, @UserID" + profile avatar
- **Summary card:** Penjualan Rp X.XXX.XXX / Pengeluaran Rp X.XXX.XXX with month selector ("Maret")
- **Module grid:** 6 cards — Master Bahan, Master Resep, HPP, Pengeluaran, Penjualan, Laporan
- **Bottom nav:** Beranda | (center FAB) | Profil

6.2 Laporan (Reports) Screen

- **Filter bar:** Date range picker, filter by Day / Month / Year
- **Income vs Expense bar chart** (fl_chart BarChart)
- **Expense breakdown pie chart** by category
- **Transaction list** filterable by date
- **Export buttons:** CSV and PDF

6.3 Backend — Report Endpoints

Method	Endpoint	Description
GET	/reports/summary?from=&to=	Aggregated income & expense
GET	/reports/by-category?from=&to=	Expense breakdown by category
GET	/reports/export/csv?from=&to=	CSV download
GET	/reports/export/pdf?from=&to=	PDF download (use pdfkit library)

6.4 PDF Export Structure

The exported PDF should include:

1. Header: BakersGo logo, bakery name, period
2. Summary table: Total Income, Total Expenses, Net Profit, Safe Balance
3. Sales detail table: Date, Product, Channel, Qty, Amount
4. Expense detail table: Date, Category, Description, Amount
5. Footer: Generated date

Install PDF library: `pnpm add pdfkit`

Agent Prompt for Step 6:

"Claude, proceed to Step 6. Build the Analytics Laporan screen in Flutter using `fl_chart`. Show a bar chart comparing income vs expenses and a pie chart for expense categories. Connect to the backend report endpoints with a date-range filter. Implement PDF export in the backend using `pdfkit`, generating a structured report with the summary and transaction detail tables. The Flutter export button should trigger a download to the device's Downloads folder."

Step 6.5 — Web App (Next.js)

Objective: Build the web version of the Baker-Go owner dashboard using Next.js 14 (App Router), Tailwind CSS, and shadcn/ui — same features as mobile, adapted for desktop/browser layout. No new backend work required; the web app consumes the same API built in Steps 2-6.

6.5.1 Scaffold the Next.js Project

```
bash

# From the baker-go/ root
pnpm create next-app@latest webapp \
  --typescript \
  --tailwind \
  --eslint \
  --app \
  --no-src-dir \
  --import-alias "@/*"

cd webapp
```

Install additional dependencies:

```
bash

pnpm add axios next-intl js-cookie
pnpm add @tanstack/react-query @tanstack/react-query-devtools
pnpm add recharts # charts (mirrors fl_chart on mobile)
pnpm add react-hook-form @hookform/resolvers zod # form validation
pnpm add lucide-react # icons
pnpm add -D @types/js-cookie

# shadcn/ui — install component by component as needed
pnpm dlx shadcn-ui@latest init
```

When `shadcn-ui init` prompts, select:

- Style: **Default**
- Base color: **Stone** (closest to the warm brown palette)
- CSS variables: **Yes**

6.5.2 Tailwind — Design Token Sync

Update `webapp/tailwind.config.ts` to mirror the Baker-Go color palette exactly:

typescript

```
import type { Config } from 'tailwindcss'

const config: Config = {
  content: ['./app/**/*.ts,tsx', './components/**/*.ts,tsx'],
  theme: {
    extend: {
      colors: {
        primary: { DEFAULT: '#A0813A', dark: '#7A5F28', light: '#D4AA6F' },
        background: '#F5F0E8',
        surface: '#FFFFFF',
        card: '#EDE8DF',
        text: {
          primary: '#2C2C2C',
          secondary: '#8A8A8A',
        },
        success: '#4CAF50',
        error: '#E53935',
        warning: '#FFA726',
      },
      borderRadius: {
        input: '12px',
        card: '16px',
      },
      fontFamily: {
        sans: ['Inter', 'system-ui', 'sans-serif'],
      },
    },
  },
  plugins: [],
}

export default config
```

Update `webapp/app/globals.css` to set background and font defaults:

css

```

@tailwind base;
@tailwind components;
@tailwind utilities;

body {
  background-color: #F5F0E8;
  color: #2C2C2C;
  font-family: 'Inter', system-ui, sans-serif;
}

```

6.5.3 i18n with next-intl

Create `webapp/messages/en.json` and `webapp/messages/id.json` using the same string keys as the Flutter ARB files. Example:

json

```

// messages/en.json
{
  "appName": "BakersGo",
  "home": "Home",
  "profile": "Profile",
  "masterIngredients": "Master Ingredients",
  "masterRecipes": "Master Recipes",
  "hpp": "HPP",
  "expenses": "Expenses",
  "sales": "Sales",
  "reports": "Reports",
  "sales_amount": "Sales",
  "expenses_amount": "Expenses",
  "safeBalance": "Safe Balance",
  "realBalance": "Real Balance",
  "signIn": "Sign In",
  "signUp": "Sign Up",
  "save": "Save",
  "cancel": "Cancel",
  "edit": "Edit",
  "delete": "Delete",
  "language": "Language",
  "errorGeneric": "Something went wrong. Please try again.",
  "emptyState": "Nothing here yet. Click + to add."
}

```

json

```
// messages/id.json
{
  "appName": "BakersGo",
  "home": "Beranda",
  "profile": "Profil",
  "masterIngredients": "Master Bahan",
  "masterRecipes": "Master Resep",
  "hpp": "HPP",
  "expenses": "Pengeluaran",
  "sales": "Penjualan",
  "reports": "Laporan",
  "sales_amount": "Penjualan",
  "expenses_amount": "Pengeluaran",
  "safeBalance": "Saldo Aman",
  "realBalance": "Saldo Nyata",
  "signIn": "Masuk",
  "signUp": "Daftar",
  "save": "Simpan",
  "cancel": "Batal",
  "edit": "Edit",
  "delete": "Hapus",
  "language": "Bahasa",
  "errorGeneric": "Terjadi kesalahan. Silakan coba lagi.",
  "emptyState": "Belum ada data. Klik + untuk menambah."
}
```

Configure next-intl in `webapp/next.config.ts`:

typescript

```
import createNextIntlPlugin from 'next-intl/plugin'

const withNextIntl = createNextIntlPlugin('./lib/i18n.ts')

export default withNextIntl({})
```

Create `webapp/lib/i18n.ts`:

typescript

```
import { getRequestConfig } from 'next-intl/server'
import { cookies } from 'next/headers'

export default getRequestConfig(async () => {
  const locale = cookies().get('locale')?.value ?? 'en'
  return {
    locale,
    messages: (await import(`../messages/${locale}.json`)).default,
  }
})
```

Language is persisted in a cookie — the user switches it from the Profile page, same as mobile.

6.5.4 API Client

Create `webapp/lib/api.ts` — a thin Axios wrapper that attaches the JWT from the cookie on every request:

```
typescript

import axios from 'axios'
import Cookies from 'js-cookie'

const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL ?? 'http://localhost:3000',
})

api.interceptors.request.use((config) => {
  const token = Cookies.get('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      Cookies.remove('token')
      window.location.href = '/login'
    }
    return Promise.reject(err)
  }
)

export default api
```

6.5.5 App Router — Page Structure

```
webapp/app/
├─ layout.tsx                ← root layout: QueryClientProvider +
                             NextIntlClientProvider
├─ (auth)/
│   ├─ login/
│   │   └─ page.tsx          ← Sign In form
│   └─ register/
│       └─ page.tsx          ← Sign Up form
└─ (dashboard)/
    ├─ layout.tsx            ← sidebar nav + top bar
    ├─ home/
    │   └─ page.tsx          ← Dashboard: summary card + 6 module grid
    ├─ master-bahan/
    │   └─ page.tsx          ← Ingredient list + add/edit modal
    ├─ master-resep/
    │   └─ page.tsx          ← Recipe list + detail view
    ├─ hpp/
    │   └─ page.tsx          ← 3-step HPP form (wizard)
    ├─ penjualan/
    │   └─ page.tsx          ← Sales log
    ├─ pengeluaran/
    │   └─ page.tsx          ← Expense log
    ├─ laporan/
    │   └─ page.tsx          ← Analytics charts + export buttons
    └─ profile/
        └─ page.tsx          ← Profile view/edit + language switcher
```

6.5.6 Reusable Component Library

Create these components under `webapp/components/ui/` and `webapp/components/shared/`:

Layout components:

- `Sidebar` — left nav with Baker-Go logo, 7 menu links, active state in primary brown
- `TopBar` — "Halo, @UserID" greeting + avatar + language toggle
- `PageShell` — wraps each page with consistent padding and heading

Data display:

- `SummaryCard` — shows Penjualan / Pengeluaran / Saldo Aman with Rp formatting
- `ModuleGrid` — 2×3 or 3×2 grid of clickable feature cards (mirrors mobile home screen)
- `DataTable` — sortable/filterable table for ingredient, recipe, transaction lists
- `EmptyState` — illustrated empty state for all list pages

Form components:

- `CurrencyInput` — number input with automatic Rp formatting
- `PercentInput` — percentage input with live HPP preview
- `ProductSelect` — searchable dropdown linking to HPP entries

Chart components (Recharts):

- `IncomeExpenseBarChart` — mirrors `fl_chart` bar chart from mobile
- `ExpensePieChart` — category breakdown pie chart

6.5.7 Dashboard Home Page

The web dashboard home (`/home`) adapts the mobile layout for a wider screen:



6.5.8 HPP Wizard (Web)

Implement the 3-zone HPP form as a step-by-step wizard using `react-hook-form` + `zod`:

- **Step 1 — Zona Dapur:** Recipe selector, batch size, kitchen costs
- **Step 2 — Zona Final:** Packaging costs
- **Step 3 — Zona Penjualan:** Margin inputs + live calculation preview table

The live preview fetches from `POST /hpp/calculate` (add this endpoint to the backend if not already present — it calculates without saving) on every margin input change using a debounced TanStack Query mutation.

6.5.9 Web App Environment Variables

Create `webapp/.env.local`:

```
env
```

```
NEXT_PUBLIC_API_URL=http://localhost:3000    # backend dev URL
# In production, update to your Vercel backend URL:
# NEXT_PUBLIC_API_URL=https://baker-go-api.vercel.app
```

6.5.10 Vercel Deployment — Web App

The web app deploys as a separate Vercel project from the same monorepo:

```
bash

cd webapp
vercel link      # link to a new Vercel project called "baker-go-webapp"
vercel env add NEXT_PUBLIC_API_URL  # set production backend URL
vercel --prod
```

Set the **Root Directory** in Vercel project settings to `webapp` so Vercel only builds the Next.js app and ignores `backend/` and `frontend/`.

Agent Prompt for Step 6.5:

"Claude, proceed to Step 6.5. Scaffold the Next.js web app inside `webapp/` at the monorepo root. Use the App Router, Tailwind CSS, and shadcn/ui. Sync the Tailwind color palette exactly to the Baker-Go design tokens: primary (`#A0813A`), background (`#F5F0E8`), card (`#EDE8DF`). Set up next-intl with `messages/en.json` and `messages/id.json` using the same string keys as the Flutter ARB files — persist the language choice in a cookie. Create the Axios API client with JWT auth from cookies and a 401 redirect guard. Scaffold all App Router pages under `(auth)/` and `(dashboard)/` with the exact folder structure from the plan. Build the Sidebar, TopBar, SummaryCard, ModuleGrid, DataTable, CurrencyInput, PercentInput, and chart components. Implement the 3-step HPP wizard using react-hook-form + zod with a live calculation preview on Step 3. All pages must be fully connected to the shared backend API — no mock data. Currency must display in Rupiah format (Rp X.XXX.XXX) throughout."

Step 7 — Final QA & Vercel Deployment

Objective: Audit all logic, perform UI QA, and go live on Vercel.

7.1 Math Audit Checklist

Before deploying, verify each formula with test data:

- ☐ `unitPrice = packagePrice / packageVolume` — test with edge cases (e.g., volume = 0 guard)
- ☐ `baseRecipeCost = Σ (amount × unitPrice)` — verify precision (use `Decimal`, not `float`)

- ☐ `totalCogs = (baseCost + allCosts) / batchSize`
- ☐ `resellerPrice = totalCogs × (1 + resellerMargin/100)`
- ☐ `safeBalance = realBalance - (electricity + tithe + capital)`
- ☐ All currency values are rounded to 2 decimal places on display, stored with full precision in DB

7.2 UI Audit Checklist

Mobile (Flutter):

- ☐ Color palette matches: Primary `#A0813A`, background `#F5F0E8`
- ☐ Input fields: rounded corners (radius 12), filled warm cream color
- ☐ Bottom navigation: Beranda/Home + center FAB + Profil/Profile
- ☐ **i18n:** All UI strings use `AppLocalizations` keys — zero hardcoded text in widgets
- ☐ **i18n:** Switch to English → all labels update correctly across every screen
- ☐ **i18n:** Switch to Bahasa Indonesia → all labels update correctly across every screen
- ☐ **i18n:** Language preference persists after app restart
- ☐ **i18n:** PDF export uses correct language labels based on `?lang=` param
- ☐ Loading states: shimmer placeholders on all list screens
- ☐ Empty states: illustrated empty state on all list screens
- ☐ Error handling: Snackbar on API failures (messages also translated)

Web (Next.js):

- ☐ Tailwind tokens match design: primary `#A0813A`, background `#F5F0E8`, card `#EDE8DF`
- ☐ Sidebar active state uses primary brown, hover uses `primaryLight`
- ☐ All forms use `CurrencyInput` for Rp fields — no raw number inputs
- ☐ HPP wizard 3-step flow works end-to-end with live preview on Step 3
- ☐ **i18n:** Language cookie switch updates all labels instantly (no page reload)
- ☐ **i18n:** Both `en.json` and `id.json` have the same keys — no missing translations
- ☐ Charts render correctly with real data (Recharts bar + pie)
- ☐ PDF/CSV export buttons trigger correct backend endpoints
- ☐ 401 responses redirect to `/login` and clear the token cookie
- ☐ Empty states shown on all list pages when no data exists
- ☐ Responsive layout works on 1280px, 1440px, and 1920px widths

7.3 Backend Deployment to Vercel

Add `vercel.json` in `backend/`:

```
json
```

```
{
  "version": 2,
  "builds": [{ "src": "src/index.ts", "use": "@vercel/node" }],
  "routes": [{ "src": "/(.*)", "dest": "src/index.ts" }]
}
```

Run Prisma migration on production (Neon):

```
bash

# DATABASE_URL and DIRECT_URL are already set in Vercel from the Neon integration
# Pull them locally first if needed:
vercel env pull .env.production.local

# Run migrations against production Neon database
npx prisma migrate deploy

# Deploy backend
vercel --prod
```

Neon note: The `DIRECT_URL` (non-pooled) is required for `prisma migrate deploy` to work correctly in production. Ensure both env vars are set in Vercel's environment variables panel under your project settings.

7.4 Flutter Production Build

Update `lib/core/constants/api_constants.dart`:

```
dart

const String kBaseUrl = 'https://baker-go-api.vercel.app'; // replace with actual URL
```

Build for Android:

```
bash

flutter build apk --release
# Output: build/app/outputs/flutter-apk/app-release.apk
```

Build for iOS:

```
bash

flutter build ipa --release
```

7.5 Next.js Web App Production Deployment

Update `webapp/.env.local` (or set in Vercel dashboard):

```
env

NEXT_PUBLIC_API_URL=https://baker-go-api.vercel.app
```

Deploy:

```
bash

cd webapp
vercel --prod
# Web app live at: https://baker-go-webapp.vercel.app
```

Final URLs after full deployment:

Service	URL
Backend API	<code>https://baker-go-api.vercel.app</code>
Web App	<code>https://baker-go-webapp.vercel.app</code>
Mobile	APK / IPA file distributed directly

Agent Prompt for Step 7:

"Claude, final Step 7. Perform a complete math audit of all HPP and Saldo Aman calculations using test data. Check that `Decimal` types are used throughout — no float arithmetic on currency values. Do a UI audit for both platforms: (1) compare each Flutter screen against the Baker-Go design reference, (2) verify the Next.js web app Tailwind tokens match exactly, (3) verify that switching language between English and Bahasa Indonesia works correctly on both mobile and web with no hardcoded text remaining, (4) confirm language preference persists after restart/reload on both platforms, and (5) test PDF export in both languages. Once verified, deploy all three workspaces to Vercel: run `prisma migrate deploy` for the backend, `vercel --prod` in `/backend`, and `vercel --prod` in `/webapp`. Update the Flutter API base URL and build the release APK."

Database Schema Reference

Complete entity-relationship overview:

```
User
├─ Ingredient[]      (Master Bahan)
├─ Recipe[]          (Master Resep)
│   └─ RecipeItem[] → Ingredient
├─ HppEntry[]        → Recipe
├─ Transaction[]     (Sales & Expenses)
└─ Expense[]
```

All user data is scoped by `userId` — every query must include a `WHERE userId = req.user.id` condition.

Design System Reference

Extracted from Baker-Go design file (12 screens):

Token	Value	Usage
<code>primary</code>	 <code>#A0813A</code>	Buttons, active icons, headings
<code>primaryDark</code>	 <code>#7A5F28</code>	Pressed state, dark header text
<code>background</code>	 <code>#F5F0E8</code>	Scaffold background
<code>cardBg</code>	 <code>#EDE8DF</code>	Input fills, summary cards
<code>surface</code>	 <code>#FFFFFF</code>	Module cards, bottom sheet
<code>textPrimary</code>	 <code>#2C2C2C</code>	Body text
<code>textSecondary</code>	 <code>#8A8A8A</code>	Labels, placeholders
Border radius	<code>12px</code>	Inputs, buttons
Border radius	<code>16px</code>	Cards, modules
Bottom nav height	<code>~72px</code>	Bottom navigation bar
FAB center	Yes	Center of bottom nav bar

Screen inventory (12 design screens):

Page	Screen Name	Route
1	Onboarding / Splash	/
2	Auth Selector	/auth
3	Sign In	/login
4	Profile (view)	/profile
5	Profile (edit)	/profile/edit
6	Sign Up (step 1)	/register
7	Sign Up (step 2)	/register
8	Welcome / Success	/welcome
9	Home / Beranda	/home
10	Change Password	/profile/password
11	Home (loaded data)	/home
12	Master Bahan (list)	/master-bahan

This document is the canonical reference for all development on Baker-Go. Architecture: Monorepo with shared Fastify backend, Flutter mobile app, and Next.js web app. Database: Neon Serverless Postgres via Vercel Marketplace (free tier). Languages: English (default) + Bahasa Indonesia — ARB files on mobile, next-intl JSON on web. Update this file as implementation details evolve.